

# SIMATS ENGINEERING ASSESSMENT TOOL |

COURSE NAME: Theory of Computation

COURSE CODE: CSA1301

---

## COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA,  $\epsilon$ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

*Assessment Tool Weightage: 10%*

---

QUESTIONS: 5

Total Marks:25

Duration : 1 Hour

## Application Based Question

---

*Code of Conduct:*

I Deepak V-1926110<sup>21</sup> (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

*I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Deepak

## SIMATS ENGINEERING ASSESSMENT TOOL

1. In modern digital systems such as embedded security devices, ATM machines, and authentication modules, password validation must be efficient and reliable. Design a **Deterministic Finite Automaton (DFA)** over the alphabet  $\Sigma = \{0,1\}$  to validate passwords that **end with the pattern "01"**. Further, **minimize the DFA** and analyze how the optimized design improves system efficiency in real-world hardware implementations

2. In web applications, validating user input is essential to ensure correctness, security, and performance. Design a regular expression to validate a specific input pattern "aaaabb" over the alphabet  $\Sigma = \{a, b\}$ . Further, analyze how regular languages and regular expressions are used to efficiently validate user inputs in real-world web applications.

3. In modern computing systems such as intrusion detection systems, compilers, and real-time monitoring tools, pattern recognition plays a critical role. Demonstrate how the regular expression  $a^*b^*$  can be converted into an equivalent finite automaton (FA). Further, explain why regular expressions and finite automata are equivalent models and how they are effectively used in real-time detection systems.

4. In compiler design, not all patterns can be recognized using finite automata. Using the pumping lemma, demonstrate why the language  $L = \{ a^n b^n \mid n \geq 0 \}$  cannot be recognized by any finite automaton. Further, analyze the implications of this limitation in the design of modern compilers.

5. Modern systems such as intrusion detection, spam filtering, and input validation often require combining multiple pattern-matching rules. Explain how the **closure properties of regular languages** enable the combination of multiple detection rules. Further, justify why the resulting language after applying these operations **remains regular**, ensuring efficient implementation using finite automata



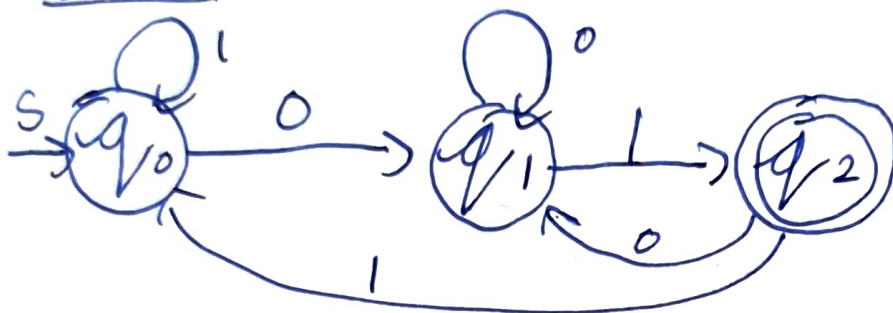
# DFA for Passwords ending with "01"

Sol:

Regular Expression:

$$RE = (a+b)^*(0+1)^*01$$

DFA:



Transition Table:

| State \ IP | 0     | 1       |
|------------|-------|---------|
| $q_0$      | $q_1$ | $q_0$   |
| $q_1$      | $q_1$ | $q_2^*$ |
| $q_2^*$    | $q_1$ | $q_2^*$ |

Minimization:

→ No dead states present

$$P_0: \{q_0, q_1, q_2\}$$

$$P_1: \{q_0, q_1, q_2\}$$

$$P_2: \{q_0, q_1, q_2\}$$

Same 3 states

∴ DFA already minimized.

Efficiency:

→ Fewer states require fewer flip-flops and logic gates

→ transition logic simpler

→ Resource constrained embedded security devices.

→ Reduces power consumption.

## 2) Regular Expression for Input Validation: Reg

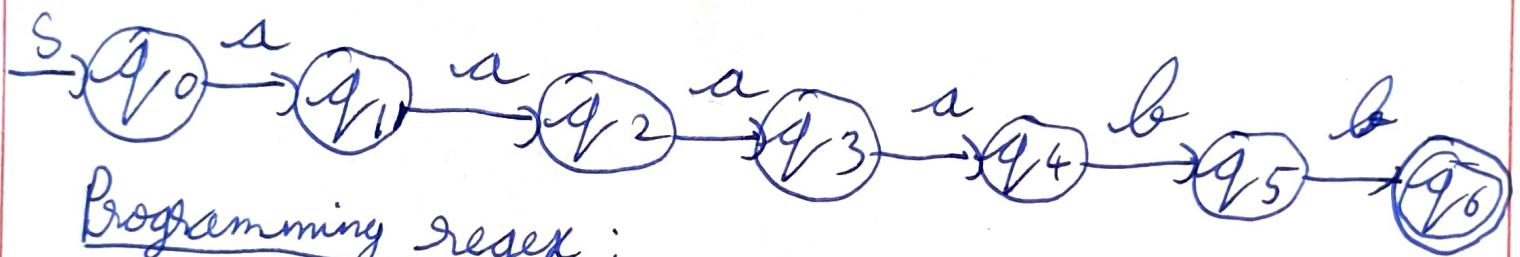
User validation is important for maintaining correctness, security and performance. Reg

Required pattern: .aaaabbb

alphabet:  $\Sigma = \{a, b\}$

Regular Expression:

RE =  $aaaabbb$  or  $a^4 b^2$



Programming regex:

$\wedge$  .aaaabbb\$

$\wedge$  -> beginning of input  
\$ -> end of input

Applications:

-> Email Validation.

-> Password Validation

-> Phone numbers

-> Form Validation

-> Security filtering

## Regular Expression to Finite Automaton

Regular expressions are used to describe patterns in strings, while finite automata are computational models used to recognize those patterns.

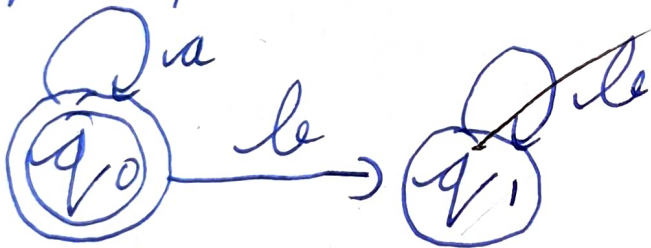
Examples:  $\epsilon$ ,  $a$ ,  $aaa$ ,  $b$ ,  $bbb$ ,  $ab$ ,  $abab$ .

Conversion of  $a^*b^*$  into Finite automaton:

$a^*b^*$

$q_0$  : processes  $a^*$

$q_1$  : processes  $b^*$



Equivalence:

They both describe exactly the same class of languages, called regular languages.

- every regex can be converted to FA and vice versa
- Both models recognize same regular languages.

#### 4) Pumping Lemma and Finite Automata

The given language is:

$$L = \{ a^n b^n \mid n \geq 0 \}$$

it contains strings with equal  $a$ 's and  $b$ 's.

Proof:

Let  $p$  be the pumping length and choose

$$xw = a^p b^p \text{ where, } |xy| \leq p, \\ |y| > 0$$

Since the first  $p$  symbols are  $a$ ,  $y$  contains only  $a$ .

$$\text{Let : } y = a^k, k > 0$$

Pump  $y$  out by taking  $i=0$ :  $xy^0z = xz$

$$\therefore xz = a^{p-k} b^p$$

Now no. of  $a$  and  $b$  different

$$xz \notin L$$

$L = \{ a^n b^n \mid n \geq 0 \}$  is not regular.

$\therefore$  No finite automation can recognize this language.

# Closure Properties of Regular Languages

Regular languages have closure properties, which means that applying certain operations to regular languages produces another regular language.

Suppose  $L_1$  and  $L_2$  are regular languages

## Important Closure Properties:

- Union:  $L_1 \cup L_2$
- Intersection:  $L_1 \cap L_2$
- Complement:  $\overline{L_1}$
- Concatenation:  $L_1 L_2$
- Kleene Star:  $L_1^*$

The resulting Language remains regular after applying these operations.

## Applications:

- Intrusion detection
- Spam Filtering
- Input Validation
- Network Monitoring
- Log Analysis.

## SIMATS ENGINEERING ASSESSMENT TOOL

### RUBRICS FOR CALCULATION

| Criteria                 | Weightage | Level 4 - Exemplary (5 Marks)                                | Level 3 - Proficient (4 Marks)         | Level 2 - Developing (2-3 Marks)             | Level 1 - Beginning (0-1 Mark)  |
|--------------------------|-----------|--------------------------------------------------------------|----------------------------------------|----------------------------------------------|---------------------------------|
| Conceptual Understanding | 30        | Demonstrates clear and complete understanding of the concept | Good understanding with minor gaps     | Basic understanding with some misconceptions | Poor or incorrect understanding |
| Accuracy of Answer       | 25        | Completely correct answer with no errors                     | Mostly correct with minor errors       | Partially correct with noticeable errors     | Incorrect or irrelevant answer  |
| Application of Knowledge | 20        | Correctly applies concepts to solve the question/problem     | Applies concepts with minor mistakes   | Limited or partially correct application     | No or incorrect application     |
| Completeness of Response | 15        | Covers all required parts of the question                    | Covers most parts with minor omissions | Covers only some parts                       | Incomplete or missing response  |
| Clarity & Presentation   | 10        | Clear, concise, and well-organized answer                    | Understandable with minor issues       | Somewhat unclear or poorly structured        | Difficult to understand         |

89/100

JF